



Construire des agents IA fiables

Comprendre la boucle avec du pseudo-code, puis vérifier ses décisions.

Vous prenez les décisions. Nous suivons leurs effets.

- | | | |
|-----------|---|---------------|
| 01 | Partir de l'alerte
Comprendre les faits et répartir les rôles. | 7 min |
| 02 | Faire un aller-retour avec l'IA
Lire un appel, exécuter un outil, renvoyer le résultat. | 20 min |
| 03 | Construire et protéger la boucle
Jouer la mission, puis provoquer deux erreurs. | 14 min |
| 04 | Vérifier et réutiliser
Lire des tests et expliquer une règle pour son métier. | 9 min |

09:42 : une clinique signale une hausse de température.

Alerte CC-204

Clinique KCARE-ADJ-01

12,4 °C depuis 52 min

Vaccins · lot VX-204

Le réfrigérateur de la clinique a déclenché une alerte. Votre équipe doit vérifier les mesures et retrouver la procédure à suivre.

Vous êtes l'équipe de garde.

Préparez un dossier justifié par des faits et une décision d'opérateur simulé.

Exercice fictif : données, procédures et opérateur simulés. Aucun acte médical ni aucune action physique.

12,4 °C : une mesure, pas une moyenne.

Ce que l'on observe

Le capteur indique 12,4 °C.
L'excursion dure 52 minutes. Son état est OK.

La règle de cet exercice

La plage de référence est 2 – 8 °C. Au-dessus de 8 °C pendant au moins 30 minutes, le cas est critique.

Ce que l'on doit faire

Consulter la procédure, qualifier le risque et obtenir la revue requise. La proposition de transfert reste simulée.

Données et règles fictives du lab : aucune intervention physique ni décision médicale réelle.

Trois rôles à garder en tête.

Le modèle propose

Il reçoit une demande et un contexte. Il propose une réponse ou un appel d'outil.

Python contrôle et exécute

Notre programme vérifie la demande, appelle la fonction et conserve son résultat.

L'opérateur décide

Pour ce cas critique, un opérateur doit approuver l'action proposée. Son rôle est simulé dans le lab.

Un appel proposé par le modèle ne constitue pas une action déjà exécutée.

Commencer par une conversation.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
demande ← « Examine l'alerte KCARE-ADJ-01 »  
historique ← [consigne, demande]  
réponse ← interroger_IA(historique, outils = [])  
afficher(réponse.texte)
```

COMMENT LE LIRE

← signifie « conserver la valeur ».

L'historique contient les messages.
Une liste vide [] signifie : aucun outil disponible.

interroger_IA est un nom pédagogique.

Exemple illustratif : « Il me faut une mesure et la procédure. » La formulation réelle peut varier.

À VOUS · L'IA peut-elle vérifier le capteur ici ?

Définir une fonction ne la lance pas.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
DÉFINIR lire_clinique(identifiant) :  
    RENVoyer données_locales[identifiant]  
  
mesure ← lire_clinique("KCARE-ADJ-01")  
afficher(mesure)
```

COMMENT LE LIRE

DÉFINIR prépare une recette.

L'appel avec un identifiant l'exécute.

La valeur renvoyée est stockée dans mesure. Ici, les données sont locales et fictives.

Après l'appel : température = 12,4 °C ; durée = 52 min ; capteur = OK.

À VOUS · À quelle ligne le travail se fait-il ?

Expliquer au modèle l'outil disponible.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
outil ← {  
  nom : "get_clinic_status",  
  description : "Lire l'état d'une clinique",  
  argument : clinic_id, texte obligatoire  
}  
outils ← [outil]
```

COMMENT LE LIRE

Un schéma décrit le nom, le rôle et les arguments attendus.

clinic_id veut dire « identifiant de clinique ».

Le développeur écrit la fonction et cette description.

Le modèle voit une description de l'outil. Python conserve la fonction à exécuter.

À VOUS · Le modèle devine-t-il nos fonctions cachées ?

Avec l'outil, la réponse peut devenir une demande.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
réponse ← interroger_IA(historique, outils)

appel proposé :
  nom = "get_clinic_status"
  arguments = {clinic_id: "KCARE-ADJ-01"}
```

COMMENT LE LIRE

Un appel contient le nom de l'outil et les valeurs à lui transmettre.

Les accolades regroupent des champs nommés.

Le modèle propose. Rien n'est encore exécuté.

Proposition illustrative du premier tour. La mesure sera connue après exécution.

À VOUS · Avons-nous déjà lu la température ?

Python retrouve la fonction et vérifie l'appel.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
SI outil inconnu OU arguments invalides :  
    arrêter avec une erreur explicite  
SINON :  
    fonction ← registre[appel.nom]  
    résultat ← fonction(appel.arguments)  
    conserver le résultat dans la trace
```

COMMENT LE LIRE

Le registre relie un nom à une fonction Python déjà écrite.

La validation contrôle les champs obligatoires et leurs types.

Cette écriture simplifie le passage des arguments.

`get_clinic_status` reçoit `clinic_id` et renvoie 12,4 °C, 52 min, capteur OK.

À VOUS · Un appel bien écrit est-il toujours cohérent ?

Renvoyer le résultat au modèle.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
ajouter(historique, proposition_du_modèle)
ajouter(historique, résultat_de_l'outil)

suite ← interroger_IA(historique, outils)
```

COMMENT LE LIRE

Le modèle ne voit pas spontanément ce qui s'est passé dans Python.

Nous ajoutons sa demande puis la réponse de l'outil, liées au même appel.

C'est un nouvel appel à l'IA.

Le nouveau contexte contient la mesure réellement lue. Le modèle peut demander la procédure.

À VOUS · Que perd-on si le résultat n'est pas ajouté ?

Prédisez le prochain outil.

Ce que nous savons

Clinique KCARE-ADJ-01 ; température 12,4 °C ; durée 52 min ; capteur OK.

Trois propositions

- A. Déclarer le stock sûr.
- B. Chercher la procédure.
- C. Inventer une température normale.

Votez, puis justifiez votre choix avec une information qui manque encore.

Construire la boucle : le modèle choisit la suite.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
RÉPÉTER au maximum 8 tours :  
  réponse ← interroger_IA(historique, outils)  
  SI réponse contient un appel :  
    exécuter avec contrôles et tracer  
    ajouter appel et résultat à l'historique  
  SINON :  
    vérifier les preuves puis conclure ou bloquer  
  TERMINER  
ARRÊTER si la limite est atteinte
```

COMMENT LE LIRE

Un tour = une nouvelle proposition du modèle.

Une observation prépare le tour suivant.

Les contrôles et l'arrêt restent sous la responsabilité du programme.

Vue simplifiée : un appel à la fois. Les erreurs et répétitions seront détaillées ensuite.

À VOUS · Qu'est-ce qui décide de la prochaine étape ?

Dans le notebook, qui est client ?

PSEUDO-CODE · À LIRE, PAS À COPIER

```
client ← créer_client(mode_choisi)

client.mode
client.complete(messages, outils)
```

COMMENT LE LIRE

client est une variable qui garde un objet.

.mode est un attribut : une valeur que l'on lit.

.complete(...) est une méthode : une fonction que l'on appelle sur cet objet.

Le contrat fourni annonce mode et complete avant leur utilisation. `selected_client` désigne le client reçu en paramètre.

À VOUS · mode et complete sont-ils tous les deux des méthodes ?

Faire le lien avec les vraies lignes Python.

PYTHON RÉEL

```
# Python réel : noms définis dans le notebook
def propose_tool(messages, selected_client):
    turn = selected_client.complete(
        messages, TOOL_SCHEMAS
    )
    calls = turn.tool_calls
    call = calls[0] if calls else None
    return turn, call
```

COMMENT LE LIRE

messages : l'historique reçu.

selected_client : le client reçu.

TOOL_SCHEMAS : les descriptions fournies.

turn : la réponse.

tool_calls : la liste d'appels proposés.

[0] : le premier ; None : aucun.

Cette cellule définit la fonction. Une cellule suivante l'appelle avec l'historique et le client choisi.

À VOUS · D'où vient `selected_client` dans cette fonction ?

Jouons la mission complète.

Question	Outil Python	Résultat attendu
Que mesure le capteur ?	<code>get_clinic_status</code>	La télémétrie de la clinique.
Quelle procédure appliquer ?	<code>search_cold_chain_sop</code>	La procédure locale retrouvée.
Que disent les règles du lab ?	<code>assess_excursion_risk</code>	La gravité et l'action proposée.
Comment documenter l'alerte ?	<code>create_incident</code>	Un identifiant de dossier.
L'opérateur approuve-t-il ?	<code>request_human_review</code>	Une décision liée à cet incident.

Le chemin dépend de la demande. Les règles d'autorisation restent dans le code.

Appliquer une règle connue, sans la réinventer.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
SI capteur indisponible :  
    risque ← INCONNU ; demander une inspection  
SINON SI température > 8 ET durée ≥ 30 :  
    risque ← CRITIQUE  
    proposition ← QUARANTINE_AND_TRANSFER  
    revue_humaine_requise ← vrai
```

COMMENT LE LIRE

Extrait des règles du lab, pas toute la fonction.

assess_excursion_risk applique ces règles déterministes.

Le modèle choisit de l'appeler ; il ne fixe pas le seuil.

12,4 > 8 et 52 ≥ 30 : le cas est CRITICAL et exige une revue. Aucun transfert réel.

À VOUS · Qui calcule ici le niveau de risque ?

Première panne : la mesure a changé en route.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
mesure_observée ← 12,4  
valeur_proposée ← 5  
  
SI valeur_proposée ≠ mesure_observée :  
    refuser l'appel  
    tracer « mesure altérée »  
    arrêter proprement
```

COMMENT LE LIRE

5 est bien un nombre : le type est valide.

Mais cette valeur ne correspond pas au résultat du capteur.

Il faut vérifier la provenance, pas seulement le format.

Le calcul du risque est refusé. Un argument numérique valide peut être factuellement faux.

À VOUS · Pourquoi une simple validation de type ne suffit-elle pas ?

Le modèle dit « résolu ». Peut-on accepter sa réponse ?

La réponse proposée

« Tout est sûr ; l'incident est résolu. »

Le modèle vient de lire la mesure, la procédure et le niveau de risque.

Ce que montre la trace

Le risque exige une revue.

Aucun dossier créé.

Aucune approbation enregistrée.

Python bloque la conclusion : `stopped / review_required`. Il manque l'accord requis.

Réponse volontairement fautive du simulateur : « Everything is safe; incident resolved. »

Autoriser la conclusion à partir des preuves.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
SI preuves insuffisantes :  
    bloquer la conclusion  
SINON SI revue requise ET accord non conforme :  
    arrêter : revue requise  
SINON :  
    rendre le compte rendu avec ses preuves
```

COMMENT LE LIRE

Accord conforme = APPROVED, pour le bon incident et la bonne action.

Les mesures et la gravité doivent aussi être cohérentes.

Une demande d'accord ne vaut pas approbation.

Cas critique approuvé : dossier escaladé avec décision enregistrée. Cela ne prouve pas un transfert physique.

À VOUS · Un accord pour un autre incident suffit-il ?

Éviter une boucle sans fin.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
SI même outil ET mêmes arguments déjà exécutés :  
    arrêter : appel répété  
SI outil en erreur :  
    tracer l'erreur puis arrêter  
SI 8 tours atteints :  
    arrêter : limite atteinte
```

COMMENT LE LIRE

Une répétition est ici un appel strictement identique.

La limite de tours borne aussi les autres boucles.

Ces arrêts sont des protections du programme.

Dans ce lab, les erreurs d'outils provoquent un arrêt contrôlé ; elles ne déclenchent pas une réparation automatique.

À VOUS · Pourquoi garder une limite même avec l'anti-répétition ?

Retrouver la preuve derrière chaque étape.

Étape	Information conservée	Ce que l'on vérifie
Mesure	12,4 °C ; 52 min ; capteur OK	Les faits viennent de l'outil.
Risque	CRITICAL ; revue requise	Les arguments reprennent les faits.
Incident	Identifiant du dossier créé	Le dossier correspond à la clinique.
Revue	Décision et action approuvée	L'accord vise ce même incident.

Exemple de parcours simulé. La trace réelle garde aussi arguments, erreurs et durée.

Un test passe quand le comportement attendu est respecté.

Scénario	Résultat attendu	Solution	Test
Alerte critique complète	human_approved	human_approved	PASS
Aucune investigation	blocked	blocked	PASS
Approbation absente	review_required	review_required	PASS
Appel identique répété	blocked	blocked	PASS

L'arrêt peut être le résultat correct : les tests vérifient aussi les refus attendus.

Extraits de la solution : dix scénarios déterministes, dont sept erreurs volontaires. Aucun appel API.

Un test compare le comportement à une attente.

PSEUDO-CODE · À LIRE, PAS À COPIER

```
POUR chaque scénario préparé :  
  résultat ← lancer_agent(scénario)  
  vérifier outils, arguments et preuves  
  vérifier arrêt ou conclusion attendue  
PASS ← toutes les vérifications sont vraies
```

COMMENT LE LIRE

Une évaluation décrit à l'avance ce qui doit arriver.

Les dix scénarios du lab utilisent un simulateur déterministe.

Ils testent les protections, pas la qualité générale de Gemini.

10/10 signifie que ces dix cas passent. Cela ne garantit ni tous les futurs cas ni les choix du modèle live.

À VOUS · Que devrait-il arriver si on retire une protection ?

Si Gemini ne répond pas, on garde le fil.

Gemini

L'IA produit de nouvelles propositions. La connexion, la clé et le service doivent fonctionner.

Mock

Un simulateur suit des réponses préparées. Il permet d'observer les mêmes contrôles ; ce n'est pas une IA en direct.

Sans notebook

On joue les rôles avec ces slides : proposition, contrôle, résultat, suite. Tout le raisonnement reste visible.

Si l'API bloque : annoncer le changement, choisir mock dans le setup, puis relancer les cellules dans l'ordre.

Le notebook prolonge ce que vous venez d'expliquer.

[La V2 et son guide d'animation](#)

Reprendre les pseudo-codes et les questions du groupe.

[Notebook solution français · ouvrir dans Colab](#)

Observer les cellules déjà complétées ; exécuter dans l'ordre.

[Notebook participant français · pour s'exercer ensuite](#)

Compléter les quatre décisions d'orchestration à votre rythme.

MERCI

Pouvez-vous expliquer un agent avec vos mots ?

Nommez un outil, son résultat et une raison de bloquer la suite.

En binôme : une personne raconte la boucle ; l'autre demande « quelle preuve ? ».



[Code et notebooks](#)

[Scannez pour ouvrir le dépôt](#)

BOSSA Chabel · Ingénieur logiciel · ENSGMM

github.com/chabelbossa · linkedin.com/in/chabel-holdo-bossa